

INDUSTRIAL AUTOMATION AND IOT- LEVEL 1

**Prepared by
Dr.S.Madhavapandian,
Design Lead,
Tessolve Semiconductor Pvt. Ltd.**

Contents

BASICS OF EMBEDDED SYSTEM.....	4
1.1. EMBEDDED SYSTEMS.....	4
1.1.1. APPLICATIONS:	4
STUDY OF MSP430	5
MSP430 LAUNCHPAD OVERVIEW	6
3.1. Introduction	6
3.1.1. Hardware Features:	7
ENERGIA PIN MAP LAUNCHPAD OVERVIEW	8
4.1. Pin Map Overview	8
ENERGIA OVERVIEW	9
5.1. Overview	9
5.2. Energia - New Project Creation.....	10
5.2.1. Opening a New Energia Sketch	10
5.2.2. Selecting a Board	10
5.2.3. Selecting the Serial Port.....	11
5.2.4. Verify the code	12
BASIC DIGITAL OUTPUT.....	14
6.1. Theory :	14
6.2. Design and Program	14
BASIC DIGITAL INPUT	17
7.1. Theory :	17
7.2. Design and Program	18
WORKING WITH SERIAL COMMUNICATION	19
8.1. Universal Asynchronous Receiver Transmitter	19
8.2. Design and Program:	19
INTERFACING ADC	21
9.1. Analog to Digital Converter (ADC) :	21
9.2. Design and Program	21
ADC Calculation :	21
INTERFACING DAC	23
10.1. Pulse Width Modulation (PWM) Module:	23
10.2.General Purpose Timer Module(GPTM):	24
10.4. Design and Program	24
BASICS OF INTERNET OF THINGS	25

11.1. INTERNET OF THINGS:	25
11.2.APPLICATIONS:	25
HTTP	26
 12.1.HTTP	 26
12.2.Design and Program:	26
UBIDOTS	32
 13.1.UBIDOTS	 32
13.2.Design and Program:	32

Basics of Embedded System

1.1. EMBEDDED SYSTEMS

An embedded system is a controller with a dedicated function within a larger mechanical or electrical system, often with real-time computing constraints. Embedded systems control many devices in common use today. 98% of all microprocessor manufactured are used in embedded systems.

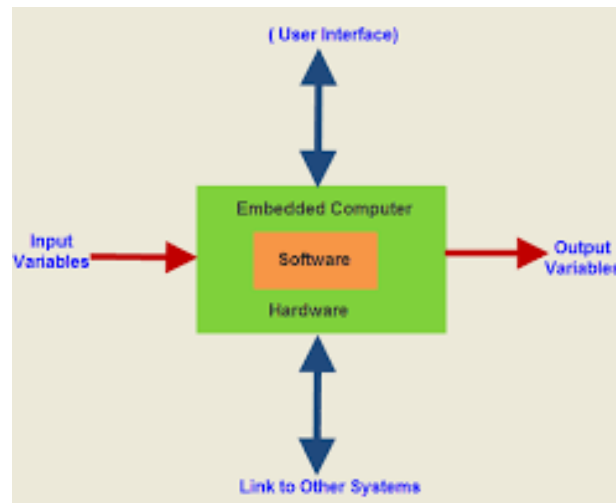


Figure 1.1. Block Diagram of Embedded Systems

Embedded systems range from portable devices such as digital watches and MP3 players, to large stationary installations like traffic lights, factory controllers, and largely complex systems like hybrid vehicles, MRI and avionics.

1.1.1. APPLICATIONS:

- Consumer application
- Smart home
- Comercial application
- Tranportation
- Building and home
- Industrial [manufacturing]
- Agriculture
- Infrastructure
- Environmental monitoring
- Living lab
- Intelligence
- Architecture
- Addressability

Study of MSP430

Simple functions like timer replacement, input/output expanders, system reset controllers and stand-alone electrically erasable programmable read-only memory (EEPROM) are common functions on printed circuit boards (PCBs). Many of these functions may not seem mission-critical, so developers typically rely on fixed-function ICs when there are other alternatives offering programmability and other enhancements.

Low-cost, ultra-low-power MSP430™ value line sensing microcontrollers (MCUs) offer cost savings when replacing digital and analog functions in a system. For example, a designer could replace five function-specific devices from different manufacturers on their board layout with five lowcost MSP430FR2000 MCUs. An increased volume of MSP430FR2000 MCUs means a lower price point per MCU and simpler purchasing and procurement of the devices for the board by using one device multiple times instead of separate devices from potentially separate vendors.

In addition to cost savings, MSP430 value line sensing MCUs are programmable, making them extremely flexible. Developers can alter control settings or operational parameters easily in software rather than triggering a hardware redesign. Plus, they can reuse a particular MCU device in other designs with either the same software, slightly altered software or completely different software. The development tools are identical, so there is no steep learning curve.

MSP430 value line sensing MCUs are supported by an ecosystem of hardware and software development tools that speed development. The provided function code examples can be modified using Code Composer Studio™ software or the IAR Embedded Workbench integrated development environments (IDEs).

MSP430 LaunchPad Overview

3.1. Introduction

The MSP-EXP430FR2433 LaunchPad™ Development Kit is an easy-to-use evaluation module (EVM) based on the MSP430FR2433 Value Line Sensing microcontroller (MCU). It contains everything needed to start developing on the ultra-low-power MSP430FR2x Value Line Sensing MCU platform, including onboard debug probe for programming, debugging, and energy measurements. The board includes two buttons and two LEDs for creating a simple user interface. It also supports using a supercapacitor (must be purchased and installed by the user) that acts like a rechargeable battery, enabling stand-alone applications without an external power supply

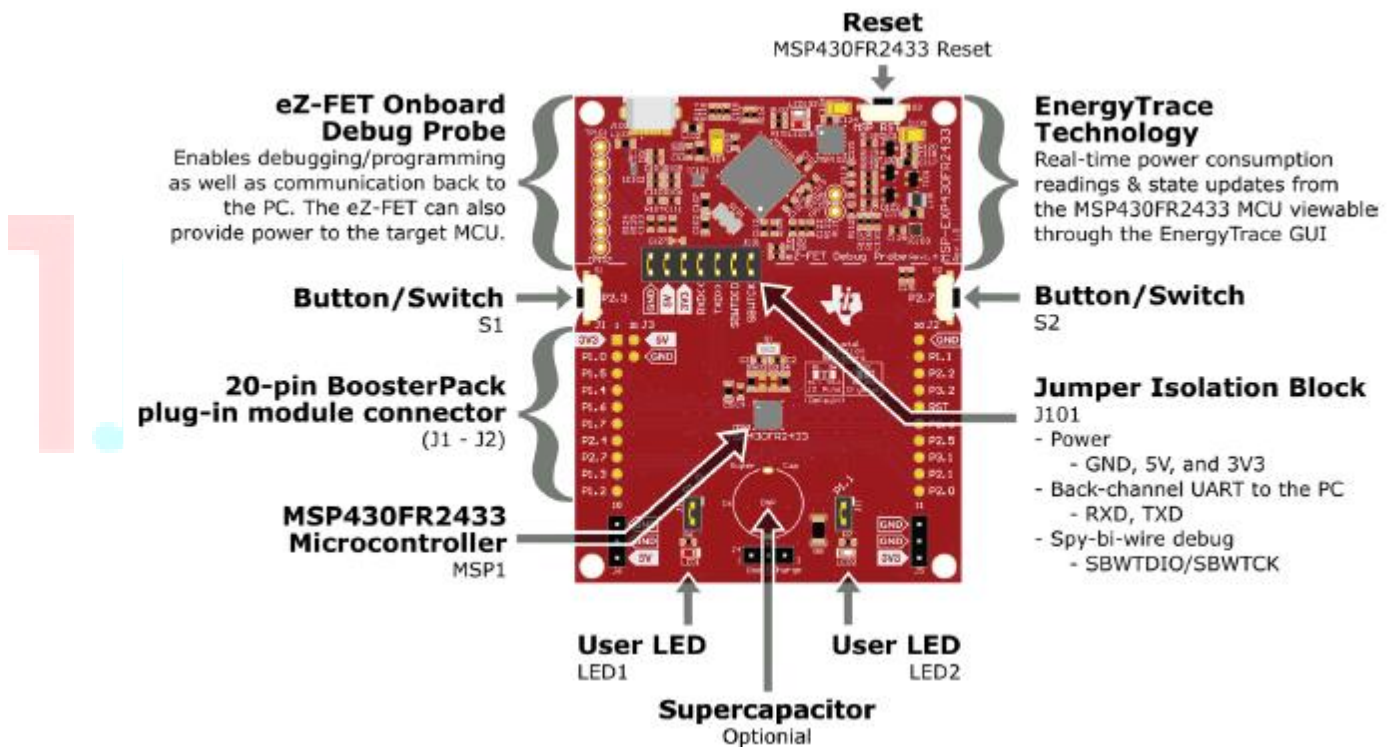


Figure 3.1 MSP430FR2433 Launchpad

The 16-MHz MSP430FR2433 device features 15.5KB of embedded FRAM (ferroelectric random access memory), a nonvolatile memory known for its ultra-low power, high endurance, and high-speed write access. Combined with the 4KB of on-chip SRAM, users have access to 15.5KB of memory to split between their program and data as required. For example, a data logging application may require a large data memory with a relatively small program memory, so the memory may be allocated as required between program and data memory. Rapid prototyping is simplified by the 20-pin BoosterPack™ plug-in module headers, which support a wide range of available BoosterPack modules. You can quickly add features like wireless connectivity, graphical displays, environmental sensing, and much more. Design your own BoosterPack plug-in module or choose among many already available from TI and third-party developers. Free software development tools are also available, such as TI's Eclipse-based Code

Composer Studio™ IDE (CCS) and IAR Embedded Workbench® IDE. Both of these IDEs support EnergyTrace™ technology for real-time power profiling and debugging when paired with the MSP430FR2433 LaunchPad development kit.

3.1.1. Hardware Features:

- 1.8-V to 3.6-V operation
- 16-bit RISC architecture up to 16-MHz system clock and 8-MHz FRAM access
- 15KB of program FRAM, 512B of information FRAM, and 4KB of RAM
- 8-channel 10-bit ADC
- Four 16-Bit timers
- Two timers with three capture/compare registers each (Timer_A3)
- Two timers with two capture/compare registers each (Timer_A2)
- 32-bit hardware multiplier (MPY)
- 19 GPIOs
- Two enhanced universal serial communication interfaces (eUSCI_A) support UART, IrDA, and SPI
- One eUSCI (eUSCI_B) supports SPI and I2C

ENERGIA Pin Map LaunchPad Overview

4.1. Pin Map Overview

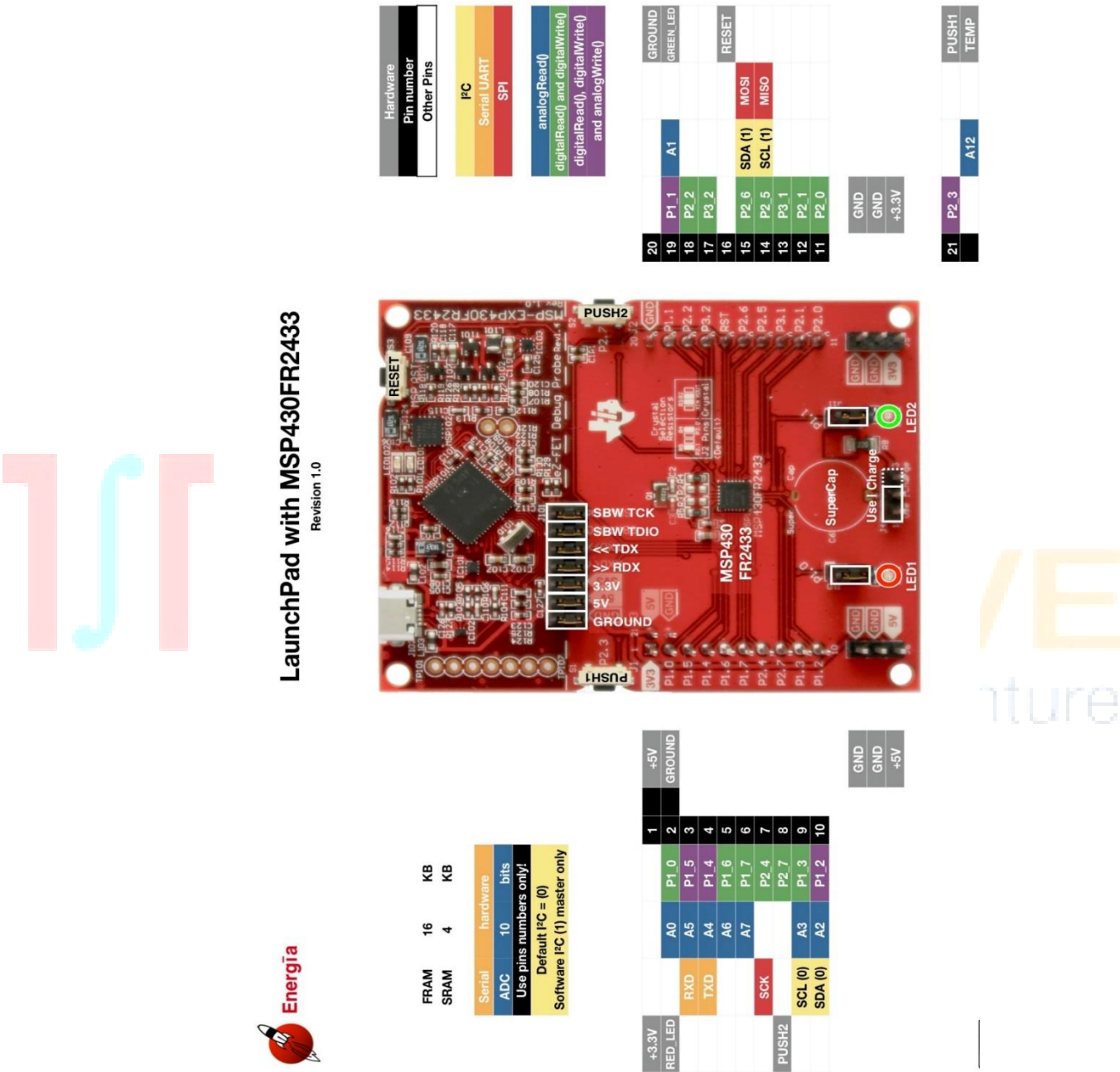


Figure 4.1. Energia Pin Map MSP430FR2433

ENERGIA Overview

5.1. Overview

Energia is an open-source electronics prototyping platform started by Robert Wessels in January of 2012 with the goal to bring the Wiring and Arduino framework to the Texas Instruments MSP430 based LaunchPad. The Energia IDE is cross platform and supported on Mac OS, Windows, and Linux. Energia uses the mspgcc compiler by Peter Bigot and is based on the Wiring and Arduino framework. Energia includes an integrated development environment (IDE) that is based on Processing. Energia is also a portable framework/abstraction layer that can be used in other popular IDEs. Utilize a web browser based environment with CCS Cloud at dev.ti.com. Community maintained Energia plug-ins and integrations are available for Xcode, Visual Studio, and Code Composer Studio.

The foundation of Energia and Arduino is the Wiring framework that is developed by Hernando Barragan. The framework is thoughtfully created with designers and artists in mind to encourage a community where both beginners and experts from around the world share ideas, knowledge and their collective experience. The Energia team adopts the philosophy of learning by doing and strives to make it easy to work directly with the hardware. Professional engineers, entrepreneurs, makers, and students can all benefit from the ease of use Energia brings to the microcontroller.

Energia started out to bring the Wiring and Arduino framework to the Texas Instruments MSP430 LaunchPad. Texas Instruments offers a MSP430, MSP432, TM4C, C2000, and CC3200 LaunchPad. The LaunchPad is a low-cost microcontroller board that is made by Texas Instruments. The latest release of Energia supports the majority of the LaunchPad product offerings. Additional community kits from RedBearLab are also supported.

5.2. Energia - New Project Creation

5.2.1. Opening a New Energia Sketch

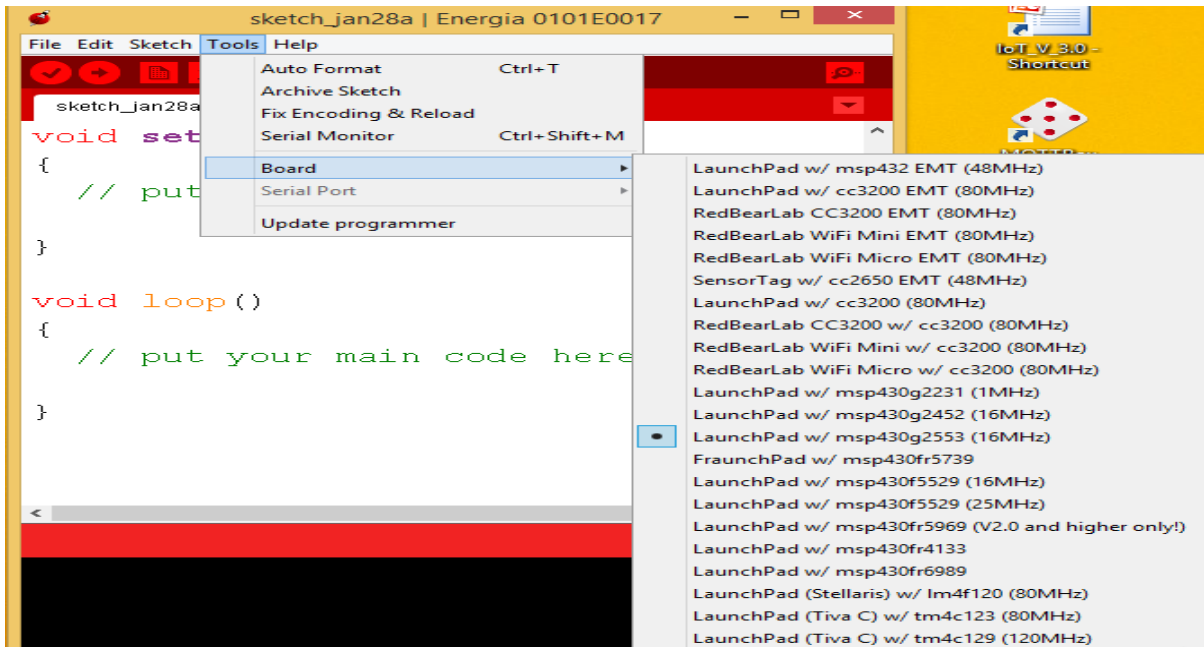
Click on the Energia icon, Following Energia sketch will open



Energia sketch consists of two loops namely `setup()` and `loop()`. The `setup()` is used for the declaring the configuration of the various pins and modules of the launchpad and the `loop()` is used for writing the code which will be executed continuously.

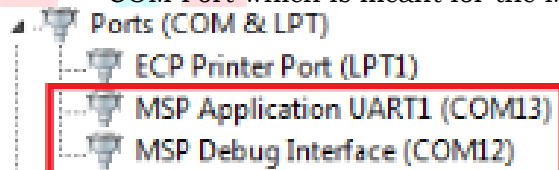
5.2.2. Selecting a Board

To select a proper board , Go to Tools->Board--> Launchpad w/mps430

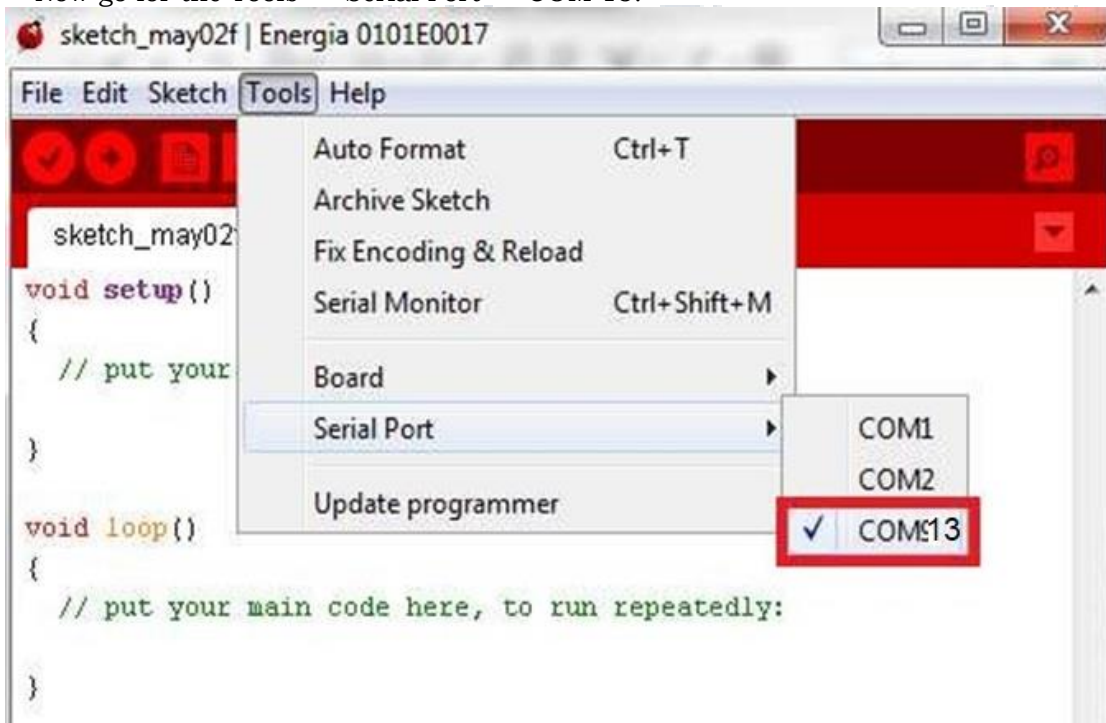


5.2.3. Selecting the Serial Port

To select a proper serial port. Go to device manager--> Ports(COM and LPT) and check for the COM Port which is meant for the MSP430, In this case it was COM port 13.



Now go for the Tools--> Serial Port--> COM 13.



5.2.4. Verify the code

To verify the code for errors and bugs, click on the Verify Icon



5.2.5. Upload the code

To upload the code on to the board, click on the Upload icon.



For CC3200 launchpad upon successful uploading of the code on the launchpad “Download Complete” message will appear as shown

```
BlockSize is 4096, number of blocks is 16  
erasing 13 blocks starting from 0  
erasing file "/sys/mcuimg.bin"  
deleting file "/sys/mcuimg.bin"  
erase file completed  
Downloading file "/sys/mcuimg.bin" with size 3000  
.Download complete
```

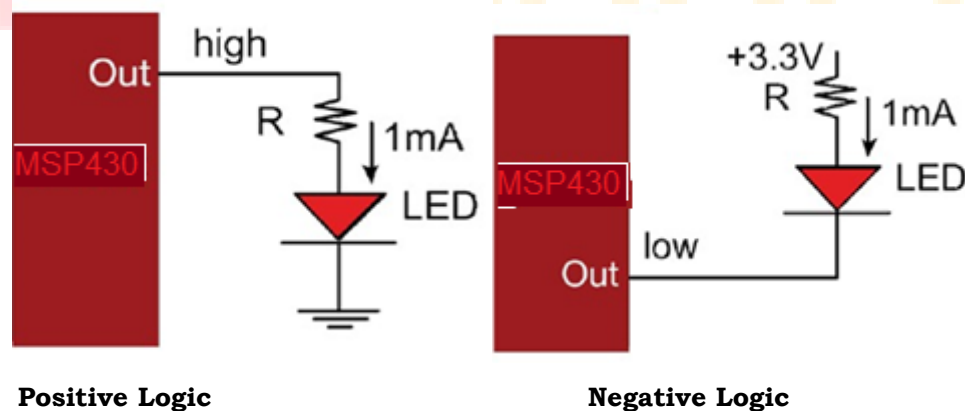
Remove the jumper from the SOP2 position and press the reset button of the launchpad for codes to run on the launchpad.

Basic Digital Output

6.1. Theory :

The pins on the LaunchPad can be configured as either inputs or outputs. Pins configured as OUTPUT with `pinMode()` are said to be in a low-impedance state. This means that they can provide a substantial amount of current to other circuits. LaunchPad pins can source (provide positive current) or sink (provide negative current) up to 40 mA (milliamps) of current to other devices/circuits. This makes them useful for powering LED's and some sensors but useless for driving motors and relays. Pins configured as outputs can also be damaged or destroyed if short circuited to either ground or 3 volt power rails. The amount of current provided by a LaunchPad pin is not enough to power many sensors, relays, or motors, and some interface circuitry will be required.

Short circuits on LaunchPad pins, or attempting to run high current devices from them, can damage or destroy the output transistors in the pin, or damage the entire TI chip. Often this will result in a “dead” pin in the microcontroller but the remaining chip will still function adequately. For this reason it is a good idea to connect OUTPUT pins to other devices with 470Ω or 1k resistors, unless maximum current draw from the pins is required for a particular application.



6.2. Design and Program

To set a pin as Output/INPUT use the following API

For setting a pin as OUTPUT

```
pinMode(PinNumber, OUTPUT);
```

To Write a data HIGH or LOW to an Energia Pin Number , use the below API

- **To write HIGH**

```
digitalWrite(PinNumber, HIGH);
```

- **To Write LOW**

```
digitalWrite(PinNumber,LOW);
```

Write a code to Turn ON and OFF the RED LED onboard MSP430 board every one second. RED LED is connected to Pin 1.0 i.e Energia pin Number 2 .

```
void setup()
{
  pinMode(2,OUTPUT);
}
void loop()
{
  digitalWrite(2,HIGH);
  delay(1000);
  digitalWrite(2,LOW);
  delay(1000);
}
```

Task 1: The digital output should be used to toggle the RED, and GREEN LED on board MSP430 Launchpad every one second.

Energia Pin Configurations:

LED Name	Energia Pin Out Details (Based on Energia Pin out diagram)
Red LED	2
Green LED	19

Energia Code for basic digital output:

```
void setup()
{
  pinMode(2,OUTPUT); pinMode(19,OUTPUT);
}
void loop()
```

```
{  
  digitalWrite(2,HIGH);  
  digitalWrite(19,HIGH);  
  delay(1000);  
  digitalWrite(2,LOW);  
  digitalWrite(19,LOW);  
  delay(1000);  
}
```



Basic Digital Input

7.1. Theory :

LaunchPad pins default to inputs, so they don't need to be explicitly declared as inputs with `pinMode()`. Pins configured as inputs are said to be in a high-impedance state. One way of explaining this is that input pins make extremely small demands on the circuit that they are sampling, say equivalent to a series resistor of 100 megohm in front of the pin. This means that it takes very little current to move the input pin from one state to another, and can make the pins useful for such tasks as implementing a capacitive touch sensor, reading an LED as a photodiode, or reading an analog sensor.

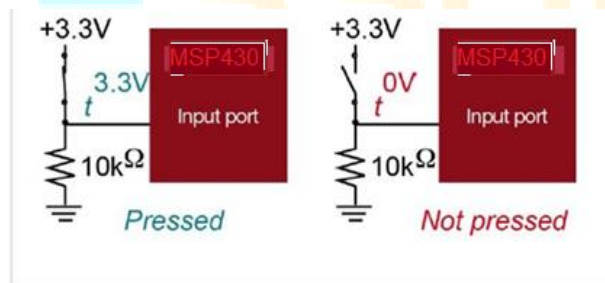
This also means however, that input pins with nothing connected to them, or with wires connected to them that are not connected to other circuits, will report seemingly random changes in pin state, picking up electrical noise from the environment, or capacitively coupling the state of a nearby pin.

For setting a pin as INPUT with PULLUP Resistor

```
pinMode(PinNumber,INPUT_PULLUP);
```

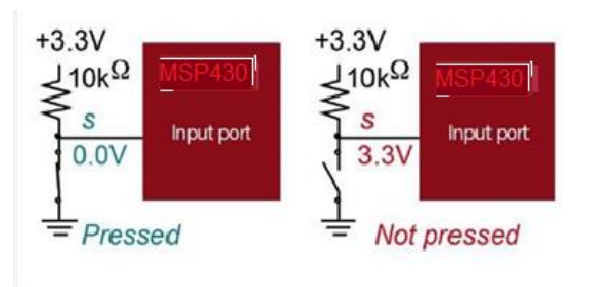
For setting a pin as INPUT otherwise

```
pinMode(PinNumber,INPUT);
```



Positive Logic *t*

- pressed, 3.3V, true
- not pressed, 0V, false



Negative Logic *s*

- pressed, 0V, true
- not pressed, 3.3V, false

7.2. Design and Program

- Write a code in Energia to turn ON Red LED when the onboard Push Button 1 switch is pressed and to turn OFF the Red LED when the Push Button 1 is released. Push Button 1 is connected to Pin Number 4 and RED LED is connected to Pin Number 29

Energia Pin Configurations:

Pin/Peripheral	Energia Pin Out Details (Based on Energia Pin out diagram)
Red LED	2
PUSH2 switch	8

Energia Code for basic digital input:

```
int buttonState = 0;

void setup()
{
  pinMode(2, OUTPUT);
  pinMode(8, INPUT_PULLUP);
}

void loop()
{
  buttonState = digitalRead(8);
  if (buttonState == HIGH)
  {
    digitalWrite(2, HIGH);
  }
  else
  {
    digitalWrite(2, LOW);
  }
}
```

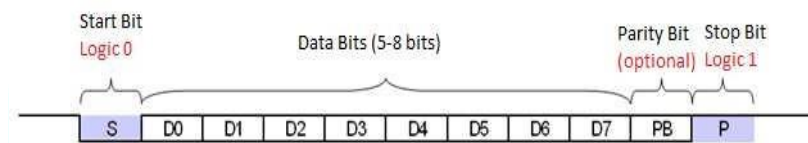
Working with Serial Communication

8.1. Universal Asynchronous Receiver Transmitter

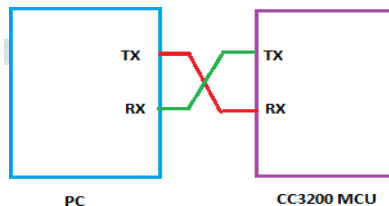
The Universal Asynchronous Receiver/Transmitter (UART) controller is the key component of the serial communications subsystem of a computer. UART is also a common integrated feature in most microcontrollers. Communication can be “full duplex” (both send and receive at the same time) or “half duplex” (devices take turns transmitting and receiving).

The UART performs serial-to-parallel conversions on data received from a peripheral device and parallel-to-serial conversion on data received from the CPU. The CPU can read the UART status at any time. The UART includes control capability and a processor interrupt system that can be tailored to minimize software management of the communications link.

UART Data Format :



UART Baud Rate : No of bits transmitted per second.



Features of UART module:

- Programmable baud-rate generator allowing speeds up to 3 Mbps.
- Separate 16×8 transmit (TX) and receive (RX) FIFOs to reduce CPU interrupt service loading. Fully programmable serial interface characteristics
 - 5, 6, 7, or 8 data bits
 - Even, odd, stick, or no-parity bit generation and detection
 - 1 or 2 stop bit generation
- System clock generates the baud clock.

8.2. Design and Program:

API to start a Serial module at a baud rate `Serial.begin(baud rate);`

API to send a data through Serial module `Serial.println(" Your message ");`

`Serial.println(variable);`

Example of available Baud rates 300, 1200, 2400, 4800, 9600, 14400, 19200, 28800, 38400, 57600, or 115200

The UART of the MSP430 in the Embedded Lab Kit is connected to the serial port of the PC that runs over the debug USB cable. The serial port can be monitored by the PC with the help of Tera Term or any other serial interface software. In this experiment, the UART of the MSP430 transmits the serial data via the USB cable. PC will receive the data and displays the received data on the serial monitor (TeraTerm) through serial communication.

UART transmission and reception pin will be available in the MSP430 launchpad as given below.

UART Signals	Energia Pin Number
TX	4
RX	3

Modify previous exercise to print a message on the serial monitor “ Button Pressed “ when the Push Button 1 is pressed and to print a message “Button Released” when the button is released.

Energia Code:

```

void setup()
{
  pinMode(2, OUTPUT);
  pinMode(8, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop()
{
  buttonState = digitalRead(8);
  if (buttonState == HIGH)
  {
    digitalWrite(2, HIGH);
    Serial.println("Pressed");
  }
  else
  {
    digitalWrite(2, LOW);
    Serial.println("Released");
  }
}

```

Interfacing ADC

9.1. Analog to Digital Converter (ADC) :

The ADC module supports fast 10-bit or 12-bit analog-to-digital conversions. The module implements a 10-bit or 12-bit SAR core together, sample select control, and a window comparator.

ADC features include:

- Greater than 200-ksps maximum conversion rate
- Monotonic 10-bit or 12-bit converter with no missing codes
- Sample-and-hold with programmable sampling periods controlled by software or timers
- Conversion initiation by software or different timers
- Software-selectable on-chip reference or external reference
- Twelve individually configurable external input channels
- Conversion channel for on-chip temperature sensor
- Selectable conversion clock source
- Single-channel, repeat-single-channel, sequence, and repeat-sequence conversion modes
- Window comparator for low-power monitoring of input signals
- Interrupt vector register for fast decoding of six ADC interrupts (ADCIFG0, ADCTOVIFG, ADCOVIFG, ADCLOIFG, ADCINIFG, ADCHIIFG)

9.2. Design and Program

The MSP430 ADC module

CHANNEL NAME	Energia Pin Out Number
ADC_CH0	2
ADC_CH1	19
ADC_CH2	10
ADC_CH3	9
ADC_CH4	4
ADC_CH5	3
ADC_CH6	5
ADC_CH7	6

ADC Calculation :

The Digital result goes up to the maximum value of 2^n . MSP430 Supports 10 bit ADC. For 10 bit ADC, ADC result = $2^{10} = 1024$. The Maximum allowable analog input voltage for ADC in MSP430 is 1.4 V. And the digital value will be calculated by using the formula

$$\text{ADC Value} = (V_{in} / V_{max}) * 2^n.$$

So the ADC result for maximum analog input voltage is = $(1.5 / 1.5) * 1024 = 1024$

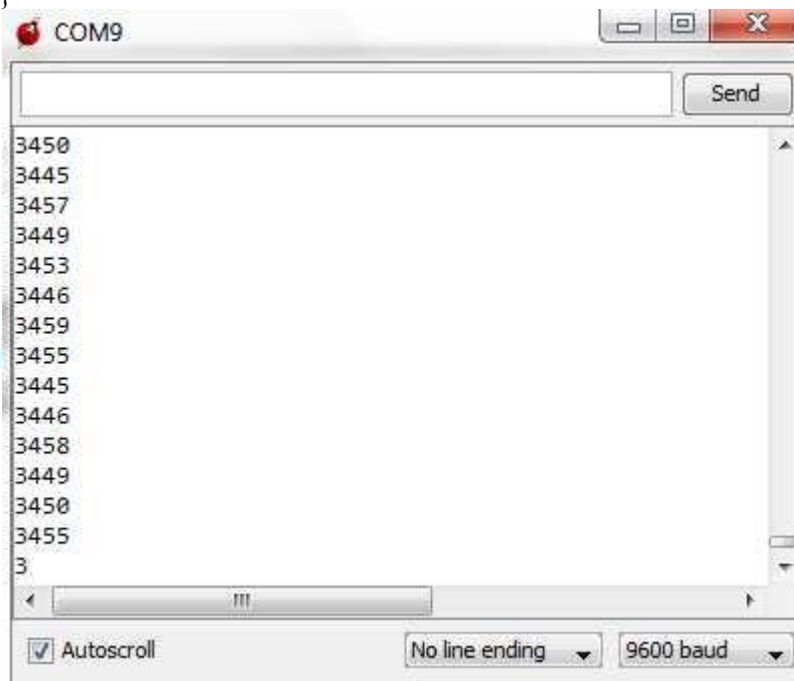
Digital values against the corresponding analog values are tabulated below.

Sl.No	Analog Input (V)	Digital Value (Decimal)
1.	0	0
2.	0.35	239

3.	0.7	478
4.	1.4	1024

Energia Code for ADC:

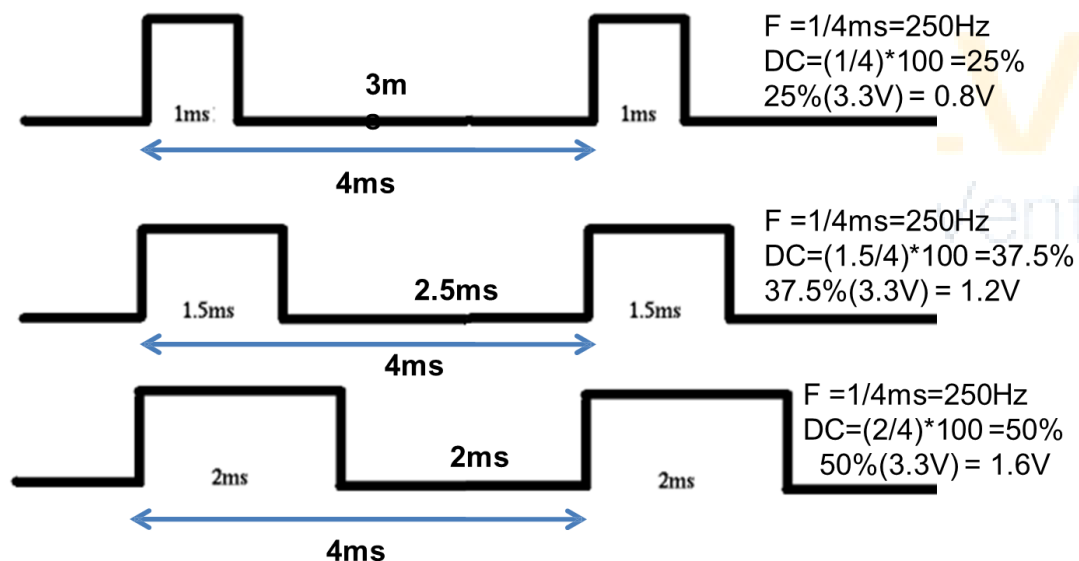
```
int analog_value=0; void setup()
{
  pinMode(2,INPUT); Serial.begin(9600);
}
void loop()
{
  analog_value=analogRead(2);
  Serial.println(analog_value);
}
```

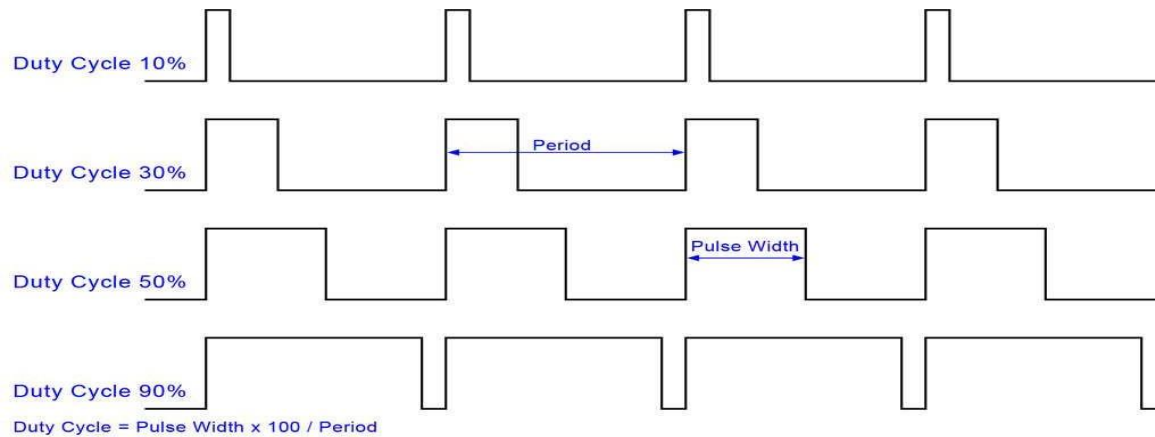


Interfacing DAC

10.1. Pulse Width Modulation (PWM) Module:

Pulse Width Modulation (PWM) is a method of digitally encoding analog signal levels. High-resolution digital counters are used to generate a square wave of a given frequency, and the duty cycle of that square wave is modulated to encode the analog signal. Typical applications for PWM are switching power supplies, motor control, servo positioning and lighting control. Using a PWM signal is one of the methods to control the brightness of the LEDs. Brightness of the LED depends on the delay in which the LED is switched ON and OFF. For example, LED is continuously switched ON for a 100% duty cycle control signal and Brightness of the LED will be the maximum. For 50% duty cycle, the LED is ON for half the time period and OFF for the remaining half period. Varying the duty cycle will vary the delay time and hence the Brightness of the LED.





10.2. General Purpose Timer Module (GPTM):

Four 16-Bit timers

- Two timers with three capture/compare registers each (Timer_A3)
- Two timers with two capture/compare registers each (Timer_A2)

Pulse Width Modulation, or PWM, is a technique for getting analog results with digital means. Digital control is used to create a square wave, a signal switched between on and off. This on-off pattern can simulate voltages in between full on (~3 Volts) and off (0 Volts) by changing the portion of the time the signal spends on versus the time that the signal spends off. The duration of "on time" is called the pulse width. To get varying analog values, you change, or modulate, that pulse width. If we repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and 3v controlling the brightness of the LED.

10.4. Design and Program

Control the brightness of the LEDs using PWM signal

Energia Code for basic digital output:

```
void setup()
{
  pinMode(19,OUTPUT);
}
void loop()
{
  for(int analog_val = 0 ; analog_val <= 255; analog_val +=5)
  {
    analogWrite(19,analog_val); delay(60);
  }
}
```


BASICS OF INTERNET OF THINGS

11.1. INTERNET OF THINGS:

The Internet of Things [IOT] is the extension of Internet connectivity into physical devices and everyday objects. Embedded with electronics, Internet connectivity, and other forms of hardware (such as sensors), these devices can communicate and interact with others over the internet, and they can be remotely monitored and controlled.

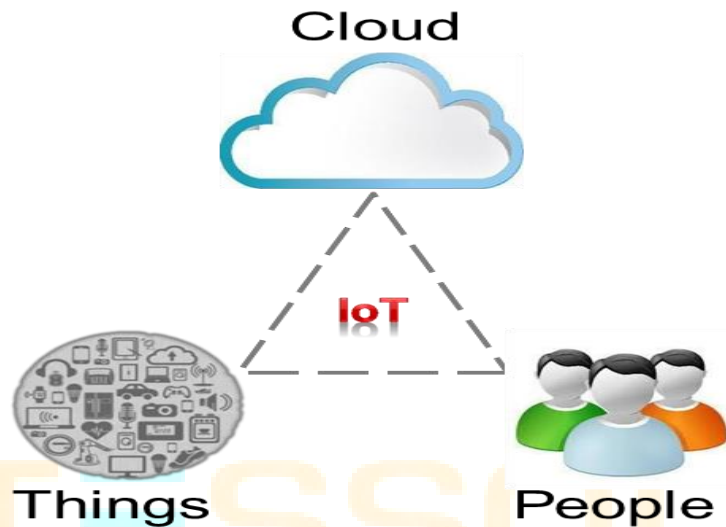


Figure 1.1. Block Diagram of IOT

The definition of Internet of Things has evolved due to the convergence of multiple technologies, real-time analytics, machine learning, commodity sensors and embedded systems.

11.2. APPLICATIONS:

- Wireless sensor networks
- Control systems
- Automation
- home automatuion
- building automation
- Covering devices
- lighting fixtures
- thermostats
- home security system and cameras.
- other home appliances.

The IOT concept faced prominent criticism, especially in regards to privacy and security concerns related to these devices and their intention of pervasive presence.

HTTP

12.1.HTTP

The Hypertext Transfer Protocol (HTTP) is the foundation of the World Wide Web, and is used to load web pages using hypertext links. HTTP is an application layer protocol designed to transfer information between networked devices and runs on top of other layers of the network protocol stack. A typical flow over HTTP involves a client machine making a request to a server, which then sends a response message.

HTTP functions as a request-response protocol in the client-server model. A web browser, for example, may be the client whereas a process, named web server, running on a computer hosting one or more websites may be the server. The client submits an HTTP request message to the server. The server, which provides resources such as HTML files and other content or performs other functions on behalf of the client, returns a response message to the client. The response contains completion status information about the request and may also contain requested content in its message body.

12.2.Design and Program:

Energia Code Webserver to Send Data:

```
#include <SPI.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <WiFiServer.h>

// your network name also called SSID
char ssid[] = "xxxx";

// your network password
char password[] = "xxxx";

// your network key Index number (needed only for WEP)
int keyIndex = 0;

WiFiServer server(80);

void setup() {
  Serial.begin(115200);    // initialize serial communication

  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to Network named: ");
  // print the network name (SSID);
```

```
Serial.println(ssid);
// Connect to WPA/WPA2 network. Change this line if using open or WEP network:
WiFi.begin(ssid, password);
while ( WiFi.status() != WL_CONNECTED) {
  // print dots while we wait to connect
  Serial.print(".");
  delay(300);
}
Serial.println("\nYou're connected to the network");
Serial.println("Waiting for an ip address");
while (WiFi.localIP() == INADDR_NONE) {
  // print dots while we wait for an ip addresss
  Serial.print(".");
  delay(300);
}
// you're connected now, so print out the status
printWifiStatus();
Serial.println("Starting webserver on port 80");
server.begin(); // start the web server on port 80
Serial.println("Webserver started!");
}

void loop() {
  // listen for incoming clients
  WiFiClient client = server.available();
  if (client) {
    Serial.println("new client");
    // an http request ends with a blank line
    boolean currentLineIsBlank = true;
    while (client.connected()) {
      if (client.available()) {
        char c = client.read();
        Serial.write(c);

        // if you've gotten to the end of the line (received a newline
        // character) and the line is blank, the http request has ended,
        // so you can send a reply
        if (c == '\n' && currentLineIsBlank) {
          // send a standard http response header
          client.println("HTTP/1.1 200 OK");
        }
      }
    }
  }
}
```

```

    client.println("Content-Type: text/html");
    client.println("Connection: close"); // the connection will be closed after completion of the
response
    client.println("Refresh: 5"); // refresh the page automatically every 5 sec
    client.println();
    client.println("<!DOCTYPE HTML>");
    client.println("<html>");
    // output the value of each analog input pin
    for (int analogChannel = 0; analogChannel < 6; analogChannel++) {
        int sensorReading = analogChannel; // analogRead(analogChannel);
        client.print("analog input ");
        client.print(analogChannel);
        client.print(" is ");
        client.print(sensorReading);
        client.println("<br />");
    }
    client.println("</html>");
    break;
}
if (c == '\n') {
    // you're starting a new line
    currentLineIsBlank = true;
}
else if (c != '\r') {
    // you've gotten a character on the current line
    currentLineIsBlank = false;
}
}
}
// give the web browser time to receive the data
delay(1);

// close the connection:
client.stop();
Serial.println("client disconnected");
}
}

```

Energia Code Webserver to Switch Red LED on/off:

```
#ifndef __CC3200R1M1RGC__
#endif
#include <WiFi.h>

// your network name also called SSID
char ssid[] = "SSID";
// your network password
char password[] = "password";
// your network key Index number (needed only for WEP)
int keyIndex = 0;

WiFiServer server(80);

void setup() {
  Serial.begin(115200);    // initialize serial communication
  pinMode(RED_LED, OUTPUT);    // set the LED pin mode
  pinMode(YELLOW_LED, OUTPUT);
  // attempt to connect to Wifi network:
  Serial.print("Attempting to connect to Network named: ");
  // print the network name (SSID);
  Serial.println(ssid);
  // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
  WiFi.begin(ssid, password);
  while ( WiFi.status() != WL_CONNECTED) {
    // print dots while we wait to connect
    Serial.print(".");
    delay(300);
  }

  Serial.println("\nYou're connected to the network");
  Serial.println("Waiting for an ip address");

  while (WiFi.localIP() == INADDR_NONE) {
    // print dots while we wait for an ip addresss
    Serial.print(".");
    delay(300);
  }

  Serial.println("\nIP Address obtained");

  // you're connected now, so print out the status
  printWifiStatus();

  Serial.println("Starting webserver on port 80");
  server.begin();          // start the web server on port 80
  Serial.println("Webserver started!");
}

void loop() {
  int i = 0;
  WiFiClient client = server.available();  // listen for incoming clients
```

```

if (client) { // if you get a client,
  Serial.println("new client"); // print a message out the serial port
  char buffer[150] = {0}; // make a buffer to hold incoming data
  while (client.connected()) { // loop while the client's connected
    if (client.available()) { // if there's bytes to read from the client,
      char c = client.read(); // read a byte, then
      Serial.write(c); // print it out the serial monitor
      if (c == '\n') { // if the byte is a newline character

        // if the current line is blank, you got two newline characters in a row.
        // that's the end of the client HTTP request, so send a response:
        if (strlen(buffer) == 0) {
          // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
          // and a content-type so the client knows what's coming, then a blank line:
          client.println("HTTP/1.1 200 OK");
          client.println("Content-type:text/html");
          client.println();

          // the content of the HTTP response follows the header:
          client.println("<html><head><title>WiFi Web Server for RED
LED</title></head><body align=center>");
          client.println("<h1 align=center><font color=\"red\">Welcome to the RED LED
Server</font></h1>");
          client.print("RED LED <button onclick=\"location.href='/H'\">HIGH</button>");
          client.println(" <button onclick=\"location.href='/L'\">LOW</button><br>");
          client.print("RED LED <button onclick=\"location.href='/YH'\">HIGH</button>");
          client.println(" <button onclick=\"location.href='/YL'\">LOW</button><br>");

          // The HTTP response ends with another blank line:
          client.println();
          // break out of the while loop:
          break;
        }
        else { // if you got a newline, then clear the buffer:
          memset(buffer, 0, 150);
          i = 0;
        }
      }
    }
    else if (c != '\r') { // if you got anything else but a carriage return character,
      buffer[i++] = c; // add it to the end of the currentLine
    }
  }

  // Check to see if the client request was "GET /H" or "GET /L":
  if (endsWith(buffer, "GET /H")) {
    digitalWrite(RED_LED, HIGH); // GET /H turns the LED on
  }
  if (endsWith(buffer, "GET /L")) {
    digitalWrite(RED_LED, LOW); // GET /L turns the LED off
  }
}
// close the connection:
client.stop();
Serial.println("client disconnected");
}
}

```

```
//  
//a way to check if one array ends with another array  
//  
boolean endsWith(char* inString, char* compString) {  
    int compLength = strlen(compString);  
    int strLength = strlen(inString);  
  
    //compare the last "compLength" values of the inString  
    int i;  
    for (i = 0; i < compLength; i++) {  
        char a = inString[(strLength - 1) - i];  
        char b = compString[(compLength - 1) - i];  
        if (a != b) {  
            return false;  
        }  
    }  
    return true;  
}  
  
void printWifiStatus() {  
    // print the SSID of the network you're attached to:  
    Serial.print("SSID: ");  
    Serial.println(WiFi.SSID());  
  
    // print your WiFi IP address:  
    IPAddress ip = WiFi.localIP();  
    Serial.print("IP Address: ");  
    Serial.println(ip);  
  
    // print the received signal strength:  
    long rssi = WiFi.RSSI();  
    Serial.print("signal strength (RSSI):");  
    Serial.print(rssi);  
    Serial.println(" dBm");  
    // print where to go in a browser:  
    Serial.print("To see this page in action, open a browser to http://");  
    Serial.println(ip);  
}
```

UBIDOTS

13.1.UBIDOTS

Ubidots is an Internet of Things (IoT) application builder with data analytics and visualization. We turn sensor data into information that matters for business-decisions, machine-to-machine interactions, educational research, and an increased economization of global resources. Hiring an engineering team to develop a platform that both functions and looks great is costly in both time and money so we did it for you. Ubidots exists as an easy and affordable means to integrate the power and value of the IoT into businesses and research. Founded in 2013, Ubidots is an investor-backed startup providing an IoT Application Enablement Platform for tinkerers, educators, and professionals.

Ubidots engineering stack delivers a secure, white-glove experience for our users with device friendly APIs (accessed over HTTP/MQTT/TCP/UDP protocols) that provide a simple and secure connection for sending and retrieving data to/from our IoT data performance optimized time-series backend (cloud). Ubidots application enablement platform supports interactive, real-time data visualization (widgets), and an IoT App Builder that allows developers to extend the platform with their own HTML/JS code for private customization when appropriate. Ubidots exists for businesses and researchers to connect devices efficiently, manage data, and economize an environment.

13.2.Design and Program:

Energia Code Ubidots data transfer:

```
#include "Ubidots.h"

const char* UBIDOTS_TOKEN = "..."; // Put here your Ubidots TOKEN
const char* WIFI_SSID = "...";    // Put here your Wi-Fi SSID
const char* WIFI_PASS = "...";    // Put here your Wi-Fi password
Ubidots ubidots(UBIDOTS_TOKEN, UBI_TCP);
void setup() { Serial.begin(115200);
  ubidots.wifiConnect(WIFI_SSID, WIFI_PASS);

  // ubidots.setDebug(true); // Uncomment this line for printing debug messages }
```



```
void loop() {  
  float value1 = random(0, 9) * 10;  
  float value2 = random(0, 9) * 100;  
  float value3 = random(0, 9) * 1000;  
  ubidots.add("Variable_Name_One", value1); // Change for your variable name  
  ubidots.add("Variable_Name_Two", value2);  
  ubidots.add("Variable_Name_Three", value3);  
  bool bufferSent = false;  
  bufferSent = ubidots.send(); // Will send data to a device label that matches the device Id  
  if (bufferSent) { // Do something if values were sent properly  
    Serial.println("Values sent by the device");  
  }  
  delay(5000);  
}
```